

Name Innocent T Muchada

Reg number H250140Q

Degree program Software Engineering

Semester level 2.1

Course name Object Orienetd Programming

Lecture Mr. Chiworera

Assignment OOP LABS

LAB1: Classes and Objects

Objectives: define classes with fields and methods; create objects with `new`; understand that variables hold references; recognise aliasing.

Basic 1 and 2: Car class with `display()` and `isAntique()`

```
public class L1_Car {
   public static void main(String[] args) {
      Car[] cars = { new Car("Toyota", 1995, 210000), new Car("Honda", 2010, 95000.5), new
Car("Ford", 1980, 320000) };
      for (Car c : cars) {
         c.display();
         System.out.println("  Antique? " + c.isAntique());
      }
   }
}
class Car {
   static final int CURRENT_YEAR = 2026;   // fixed so output is repeatable
   String brand; int year; double mileage;
   Car(String brand, int year, double mileage) { this.brand = brand; this.year = year; this.mileage
= mileage; }
   void display() { System.out.println("Brand: " + brand + ", Year: " + year + ", Mileage: " +
mileage + " km"); }
   boolean isAntique() { return CURRENT_YEAR - year > 25; }   // more than 25 years old
}
```

Output

```
Brand: Toyota, Year: 1995, Mileage: 210000.0 km
  Antique? true
Brand: Honda, Year: 2010, Mileage: 95000.5 km
  Antique? false
Brand: Ford, Year: 1980, Mileage: 320000.0 km
  Antique? true

Process finished with exit code 0
```

Intermediate 1 and 2: Rectangle, largest area, and aliasing

```
public class L1_Rectangle {
   public static void main(String[] args) {
      Rectangle[] rects = new Rectangle[5];
      double[][] dims = {{2, 3}, {5, 4}, {1, 9}, {6, 6}, {3, 7}};
      for (int i = 0; i < rects.length; i++) rects[i] = new Rectangle(dims[i][0], dims[i][1]);
```

```
        Rectangle largest = rects[0];
        for (Rectangle r : rects) if (r.area() > largest.area()) largest = r;
        System.out.println("Largest: " + largest.width + " x " + largest.height
                + ", area = " + largest.area() + ", perimeter = " + largest.perimeter());

        // Alias demonstration (Intermediate 2)
        Rectangle r1 = new Rectangle(4, 5);
        Rectangle r2 = r1;        // copies the reference, NOT the object
        r2.width = 10;
        System.out.println("r1.width = " + r1.width);
    }
}
class Rectangle {
    double width, height;
    Rectangle(double w, double h) { width = w; height = h; }
    double area() { return width * height; }
    double perimeter() { return 2 * (width + height); }
}
```

Output

```
"C:\Program Files\Java\jdk-26.0.2.1\bin\java.exe"
Largest: 6.0 x 6.0, area = 36.0, perimeter = 24.0
r1.width = 10.0

Process finished with exit code 0
```

Explain (Intermediate 2). I predicted `r1.width` would print 10.0, and it did. `Rectangle r2 = r1;` copies the reference (the address), not the object, so `r1` and `r2` are two names for the same object on the heap. Changing `r2.width` changes the one shared object, which `r1` also sees. To get an independent copy you must create a new object, for example with a copy constructor.

Conclusion. Classes define state and behaviour, objects are instances, and object variables hold references. Assignment between references creates aliases rather than copies.

LAB2: Constructors, this and static

Objectives: overload and chain constructors; use `this`; distinguish static from instance members; understand initialiser blocks.

Basic 1 and 2: Circle with two constructors and a static counter

```
public class L2_Circle {
    public static void main(String[] args) {
        Circle a = new Circle(2.5);
        Circle b = new Circle();        // default radius 1.0
        System.out.println("a radius = " + a.getRadius() + ", b radius = " + b.getRadius());
```

```
        System.out.println("Circles created: " + Circle.getCount());
    }
}
class Circle {
    private static int count = 0;       // one shared counter
    private final double radius;
    Circle(double radius) { this.radius = radius; count++; }
    Circle() { this(1.0); }             // chaining: count++ happens once, in the other constructor
    double getRadius() { return radius; }
    static int getCount() { return count; }
}
```

Output

```
"C:\Program Files\Java\jdk-26.0.2.1\bin\java.exe"
a radius = 2.5, b radius = 1.0
Circles created: 2
```

The no-argument constructor calls `this(1.0)` rather than incrementing the counter itself, so each circle is counted exactly once.

Intermediate 1: Employee with auto-id, three overloaded constructors and a copy constructor

```
public class L2_Employee {
    public static void main(String[] args) {
        Employee e1 = new Employee("Tendai", 2500, "IT");
        Employee e2 = new Employee("Rudo", 2100);
        Employee e3 = new Employee("Farai");
        Employee e4 = new Employee(e1);   // copy constructor: new id, same data
        e1.show(); e2.show(); e3.show(); e4.show();
    }
}
class Employee {
    private static int nextId = 1;       // auto-incrementing id source
    private final int employeeId;
    private final String name, dept;
    private final double salary;
    Employee(String name, double salary, String dept) {
        this.employeeId = nextId++; this.name = name; this.salary = salary; this.dept = dept;
    }
    Employee(String name, double salary) { this(name, salary, "General"); }
    Employee(String name) { this(name, 1000.0); }
    Employee(Employee o) { this(o.name, o.salary, o.dept); }   // copy constructor chains to the
full one
```

```
    void show() { System.out.printf("#%d %s, %s, %.2f%n", employeeId, name, dept, salary); }
}
```

Output

```
"C:\Program Files\Java\jdk-26.0.2.1\bin\java.exe"
#1 Tendai, IT, 2500.00
#2 Rudo, General, 2100.00
#3 Farai, General, 1000.00
#4 Tendai, IT, 2500.00
```

The copy (#4) has the same data as #1 but a new id, because it chains to the full constructor, which takes the next id.

Intermediate 2: Two static blocks and two instance blocks

```
public class L2_InitBlocks {
    static { System.out.println("static block 1"); }
    { System.out.println("instance block 1"); }
    L2_InitBlocks() { System.out.println("constructor"); }
    static { System.out.println("static block 2"); }
    { System.out.println("instance block 2"); }
    public static void main(String[] args) {
        System.out.println("main starts");
        new L2_InitBlocks();
        System.out.println("--- second object ---");
        new L2_InitBlocks();
    }
}
```

Prediction: both static blocks run once, in written order, when the class loads (before `main` starts). For each object, the instance blocks run in written order, then the constructor body.

Output (matches the prediction)

```
"C:\Program Files\Java\jdk-26.0.2.1\bin\java.exe"
static block 1
static block 2
main starts
instance block 1
instance block 2
constructor
--- second object ---
instance block 1
instance block 2
constructor
```

Conclusion. Constructors can be overloaded and chained with `this(...)`. Static members belong to the class and are shared. Static blocks run once at class load, and instance blocks run before the constructor body for every object.

LAB3: Encapsulation

Objectives: private fields with controlled access; validation in setters and constructors; avoid leaking mutable internals.

Basic 1 and 2: Student with validated marks and a derived grade

```
public class L3_StudentMarks {
   public static void main(String[] args) {
      Student s = new Student("Chipo", 72);
      System.out.println(s.getName() + ": " + s.getMarks() + " -> grade " + s.getGrade());
      s.setMarks(48);
      System.out.println(s.getName() + ": " + s.getMarks() + " -> grade " + s.getGrade());
      try {
         s.setMarks(120);
      } catch (IllegalArgumentException e) {
         System.out.println("Rejected: " + e.getMessage());
      }
      System.out.println("Marks unchanged: " + s.getMarks());
   }
}
class Student {
   private String name;
   private int marks;
   Student(String name, int marks) { this.name = name; setMarks(marks); }   // constructor
reuses validation
   String getName() { return name; }
   int getMarks() { return marks; }
```

```
    void setMarks(int marks) {
        if (marks < 0 || marks > 100) throw new IllegalArgumentException("Marks must be 0-100,
got " + marks);
        this.marks = marks;
    }
    // Derived from marks each time, so it can never get out of sync
    String getGrade() { return marks >= 70 ? "A" : marks >= 60 ? "B" : marks >= 50 ? "C" : "F"; }
}
```

Output

```
"C:\Program Files\Java\jdk-26.0.2.1\bin\java.exe"
Chipo: 72 -> grade A
Chipo: 48 -> grade F
Rejected: Marks must be 0-100, got 120
Marks unchanged: 48
```

Grade scale used: A 70–100, B 60–69, C 50–59, F below 50. The grade is calculated by `getGrade()` instead of stored in a field, so it cannot disagree with the marks.

Intermediate 1: BankAccount with a read-only transaction history

```
import java.util.*;
public class L3_BankHistory {
    public static void main(String[] args) {
        Account acc = new Account("ACC-001", 500);
        acc.deposit(250);
        acc.withdraw(100);
        acc.withdraw(5000);
        List<String> h = acc.getHistory();
        h.forEach(System.out::println);
        try {
            h.clear();                  // tampering attempt
        } catch (UnsupportedOperationException e) {
            System.out.println("History is read-only");
        }
        System.out.println("Entries still stored: " + acc.getHistory().size());
    }
}
class Account {
    private final String number;
    private double balance;
    private final List<String> history = new ArrayList<>();
    Account(String number, double opening) {
        this.number = number; this.balance = opening;
```

```
        history.add("OPEN " + opening);
    }
    void deposit(double amt) {
        if (amt <= 0) throw new IllegalArgumentException("Deposit must be positive");
        balance += amt; history.add("DEPOSIT " + amt);
    }
    boolean withdraw(double amt) {
        if (amt <= 0) throw new IllegalArgumentException("Withdrawal must be positive");
        if (amt > balance) { history.add("DECLINED " + amt); return false; }
        balance -= amt; history.add("WITHDRAW " + amt); return true;
    }
    // Returns an unmodifiable COPY: later changes to the account do not alter a list already
handed out
    List<String> getHistory() { return List.copyOf(history); }
}
```

Output

```
"C:\Program Files\Java\jdk-26.0.2.1\bin\java.exe"
OPEN 500.0
DEPOSIT 250.0
WITHDRAW 100.0
DECLINED 5000.0
History is read-only
Entries still stored: 4
```

Intermediate 2: ImmutablePerson

```
import java.util.*;
public class L3_ImmutablePerson {
    public static void main(String[] args) {
        List<String> hobbies = new ArrayList<>(List.of("chess", "football"));
        ImmutablePerson p = new ImmutablePerson("Anesu", 20, hobbies);
        hobbies.add("poker");                   // changing the original list...
        System.out.println(p);                  // ...does not affect p
        try { p.getHobbies().add("x"); }
        catch (UnsupportedOperationException e) { System.out.println("getHobbies() is read-only");
}
        ImmutablePerson older = p.withAge(21);      // "changes" create a new object
        System.out.println(p.getAge() + " vs " + older.getAge());
    }
}
final class ImmutablePerson {                   // final class: no subclass can add mutability
    private final String name;                  // final fields: assigned once
    private final int age;
```

```
    private final List<String> hobbies;
    ImmutablePerson(String name, int age, List<String> hobbies) {
        this.name = name; this.age = age;
        this.hobbies = List.copyOf(hobbies);       // defensive copy on the way in
    }
    String getName() { return name; }              // no setters at all
    int getAge() { return age; }
    List<String> getHobbies() { return hobbies; }   // already unmodifiable
    ImmutablePerson withAge(int newAge) { return new ImmutablePerson(name, newAge,
hobbies); }
    @Override public String toString() { return name + " (" + age + ") " + hobbies; }
}
```

Output

```
"C:\Program Files\Java\jdk-26.0.2.1\bin\java.exe"
Anesu (20) [chess, football]
getHobbies() is read-only
20 vs 21
```

Explain each design choice.

- `final class`: stops a subclass from adding mutable state or overriding methods to break immutability.
- `private final` fields: each is assigned once in the constructor and cannot be reassigned.
- No setters: nothing can change the object after construction.
- `List.copyOf` in the constructor: the caller keeps their own list, and changing it later (adding "poker") does not affect the person.
- Unmodifiable list returned by the getter: callers cannot change the internal list through it.
- `withAge`: "changing" a value returns a new object and leaves the original intact.

Conclusion. Private fields plus validated setters keep objects in a valid state. Returned collections must be copied or wrapped so that getters do not leak the internals.

LAB4: Inheritance

Objectives: use `extends` and `super`; see constructor order; understand why a missing superclass constructor fails to compile.

Basic 1 and 2: Person, Student and Teacher sharing one display()

```
public class L4_Person {
    public static void main(String[] args) {
        Student s = new Student("Tariro", 19, "Computer Science");
        Teacher t = new Teacher("Mr Moyo", 45, "Mathematics");
        s.display();     // display() is written once, in Person
```

```
        t.display();
        s.showCourse();
        t.showSubject();
    }
}
class Person {
    protected String name; protected int age;
    Person(String name, int age) { this.name = name; this.age = age; }
    void display() { System.out.println("Name: " + name + ", Age: " + age); }
}
class Student extends Person {
    private String course;
    Student(String name, int age, String course) { super(name, age); this.course = course; }
    void showCourse() { System.out.println(name + " studies " + course); }
}
class Teacher extends Person {
    private String subject;
    Teacher(String name, int age, String subject) { super(name, age); this.subject = subject; }
    void showSubject() { System.out.println(name + " teaches " + subject); }
}
```

Output

```
"C:\Program Files\Java\jdk-26.0.2.1\bin\java.exe"
Name: Tariro, Age: 19
Name: Mr Moyo, Age: 45
Tariro studies Computer Science
Mr Moyo teaches Mathematics
```

`Student` and `Teacher` both reuse the name, age and `display()` code written once in `Person`, and each adds only what is specific to it.

Intermediate 1: Vehicle → Car → ElectricCar (multilevel) and constructor order

```
public class L4_Vehicle {
    public static void main(String[] args) {
        System.out.println("Creating an ElectricCar:");
        ElectricCar e = new ElectricCar("ZW-123", 4, 75.0);
        System.out.println("Done: " + e);
    }
}
class Vehicle {
    protected String plate;
    Vehicle(String plate) { this.plate = plate; System.out.println("1. Vehicle constructor"); }
}
```

```
class Car extends Vehicle {
   protected int doors;
   Car(String plate, int doors) { super(plate); this.doors = doors; System.out.println("2. Car
constructor"); }
}
class ElectricCar extends Car {
   private double batteryKwh;
   ElectricCar(String plate, int doors, double kwh) { super(plate, doors); batteryKwh = kwh;
System.out.println("3. ElectricCar constructor"); }
   @Override public String toString() { return plate + ", " + doors + " doors, " + batteryKwh + "
kWh"; }
}
```

Output

```
"C:\Program Files\Java\jdk-26.0.2.1\bin\java.exe"
Creating an ElectricCar:
1. Vehicle constructor
2. Car constructor
3. ElectricCar constructor
Done: ZW-123, 4 doors, 75.0 kWh
```

The constructors *start* in the order ElectricCar → Car → Vehicle, because each begins with `super(...)`. Each superclass body therefore *finishes* first, so the messages print from the top of the hierarchy down.

Intermediate 2: Why `class Dog extends Animal` fails when `Animal` only has `Animal(String name)`

```
class Animal {
   String name;
   Animal(String name) { this.name = name; }   // only constructor: NO no-arg constructor exists
}
class Dog extends Animal {
   Dog() { }                          // implicit super() -> Animal() does not exist
}
```

Compiler output

```
Dog.java:6: error: constructor Animal in class Animal cannot be applied to given types;
   Dog() { }
       ^
  required: String
  found:    no arguments
  reason: actual and formal argument lists differ in length
1 error
```

Explain. Every subclass constructor must call a superclass constructor first. If you do not write `super(...)`, Java inserts `super()` with no arguments. Because `Animal` defines its own constructor, Java no longer supplies the default no-argument one, so `super()` has nothing to call and compilation fails. **Fix:** call the constructor that exists, for example `Dog(String name) { super(name); }`, or add a no-argument constructor to `Animal`.

Conclusion. Inheritance models an "is-a" relationship and reuses code. Superclass constructors always run first, and the subclass must call one that exists.

LAB5: Polymorphism

Objectives: overloading (compile time) versus overriding (run time); dynamic dispatch; what is *not* polymorphic.

Basic 1: Overloaded `print` for `int`, `double`, `String` and `int[]`

```
import java.util.Arrays;
public class L5_Overload {
   static void print(int x)     { System.out.println("int: " + x); }
   static void print(double x)  { System.out.println("double: " + x); }
   static void print(String x)  { System.out.println("String: " + x); }
   static void print(int[] x)   { System.out.println("int[]: " + Arrays.toString(x)); }
   public static void main(String[] args) {
      print(7); print(3.14); print("hello"); print(new int[]{1, 2, 3});
      print('A');   // char is widened to int, so print(int) is chosen
   }
}
```

Output

```
"C:\Program Files\Java\jdk-26.0.2.1\bin\java.exe"
int: 7
double: 3.14
String: hello
int[]: [1, 2, 3]
int: 65
```

The compiler picks the version from the argument types. A `char` is widened to `int`, so `'A'` prints 65.

Basic 2: Animal with `sound()` and Cat, Cow, Duck

```
public class L5_Animals {
   public static void main(String[] args) {
      Animal[] zoo = { new Cat(), new Cow(), new Duck() };
      for (Animal a : zoo) System.out.println(a.getClass().getSimpleName() + " says " + a.sound());
   }
}
```

```
class Animal { String sound() { return "..."; } }
class Cat extends Animal  { @Override String sound() { return "Meow"; } }
class Cow extends Animal  { @Override String sound() { return "Moo"; } }
class Duck extends Animal { @Override String sound() { return "Quack"; } }
```

Output

```
"C:\Program Files\Java\jdk-26.0.2.1\bin\java.exe"
Cat says Meow
Cow says Moo
Duck says Quack
```

Intermediate 1: Payroll with a new `Intern` class, loop unchanged

```
public class L5_Payroll {
   // This method never mentions Intern: it works for any Employee subclass
   static double totalPayroll(Employee[] staff) {
      double total = 0;
      for (Employee e : staff) { e.describe(); total += e.calculatePay(); }
      return total;
   }
   public static void main(String[] args) {
      Employee[] staff = { new Employee("Kudzai", 2000), new Manager("Nyasha", 3500, 800),
                           new Developer("Tatenda", 3000, 10), new Intern("Lynn", 600) };
      System.out.printf("Total payroll: %.2f%n", totalPayroll(staff));
   }
}
class Employee {
   private final String name; protected double baseSalary;
   Employee(String name, double baseSalary) { this.name = name; this.baseSalary = baseSalary; }
   double calculatePay() { return baseSalary; }
   void describe() { System.out.printf("%-8s earns %8.2f%n", name, calculatePay()); }
}
class Manager extends Employee {
   private final double bonus;
   Manager(String n, double s, double bonus) { super(n, s); this.bonus = bonus; }
   @Override double calculatePay() { return super.calculatePay() + bonus; }
}
class Developer extends Employee {
   private final int overtime;
   Developer(String n, double s, int overtime) { super(n, s); this.overtime = overtime; }
   @Override double calculatePay() { return baseSalary + overtime * 25.0; }
}
class Intern extends Employee {                  // the NEW subclass
   Intern(String n, double stipend) { super(n, stipend); }
```

```
    @Override double calculatePay() { return baseSalary * 0.9; }   // 10% training deduction
}
```

Output

```
"C:\Program Files\Java\jdk-26.0.2.1\bin\java.exe"
Kudzai   earns  2000.00
Nyasha   earns  4300.00
Tatenda  earns  3250.00
Lynn     earns   540.00
Total payroll: 10090.00
```

`totalPayroll` was written before `Intern` existed and needed no change, because dynamic dispatch picks each object's own `calculatePay()` at run time.

Intermediate 2: Field versus method accessed through a superclass reference

```
public class L5_FieldVsMethod {
    public static void main(String[] args) {
        Parent p = new Child();              // declared type Parent, real object Child
        System.out.println("Field:  " + p.label);   // follows the DECLARED type
        System.out.println("Method: " + p.who());   // follows the ACTUAL object
        System.out.println("Static: " + Parent.kind() + " / " + Child.kind());
        Child c = (Child) p;
        System.out.println("Via Child reference, field: " + c.label);
    }
}
class Parent {
    String label = "Parent field";
    String who() { return "Parent method"; }
    static String kind() { return "Parent static"; }
}
class Child extends Parent {
    String label = "Child field";            // hides, does not override
    @Override String who() { return "Child method"; }
    static String kind() { return "Child static"; }
}
```

Output

```
"C:\Program Files\Java\jdk-26.0.2.1\bin\java.exe"
Field:  Parent field
Method: Child method
Static: Parent static / Child static
Via Child reference, field: Child field
```

Explain. The method call is resolved at run time from the real object (`Child`), so `who()` prints "Child method" (dynamic dispatch). A field is resolved at compile time from the declared type of the variable (`Parent`), so `p.label` gives "Parent field". A field in a subclass only *hides* the parent's field, it does not override it. Static methods follow the class name or declared type in the same way. Only instance methods are polymorphic.

Conclusion. Overloading is chosen by the compiler from the argument types, and overriding is chosen by the JVM from the object's real class. Polymorphism lets code written once work with subclasses added later.

LAB6: Abstraction, Abstract Classes and Interfaces

Objectives: abstract classes and methods; interfaces for shared capabilities; `Comparable` and `Comparator`.

Basic 1: Abstract Shape with Circle, Rectangle and Square

```
public class L6_Shapes {
   public static void main(String[] args) {
      Shape[] shapes = { new Circle(5), new Rectangle(4, 6), new Square(3) };
      for (Shape s : shapes)
         System.out.printf("%-9s area = %7.2f  perimeter = %6.2f%n",
s.getClass().getSimpleName(), s.area(), s.perimeter());
   }
}
abstract class Shape {
   abstract double area();
   abstract double perimeter();
}
class Circle extends Shape {
   private final double r;
   Circle(double r) { this.r = r; }
   double area() { return Math.PI * r * r; }
   double perimeter() { return 2 * Math.PI * r; }
}
class Rectangle extends Shape {
   protected final double w, h;
   Rectangle(double w, double h) { this.w = w; this.h = h; }
   double area() { return w * h; }
```

```
    double perimeter() { return 2 * (w + h); }
}
class Square extends Rectangle { Square(double side) { super(side, side); } }   // reuses
Rectangle's maths
```

Output

```
"C:\Program Files\Java\jdk-26.0.2.1\bin\java.exe"
Circle    area =   78.54  perimeter =  31.42
Rectangle area =   24.00  perimeter =  20.00
Square    area =    9.00  perimeter =  12.00
```

Basic 2: Flyable interface for unrelated classes

```
public class L6_Flyable {
    public static void main(String[] args) {
        Flyable[] things = { new Bird("Eagle"), new Aeroplane("Boeing 737") };   // unrelated classes,
one type
        for (Flyable f : things) f.fly();
    }
}
interface Flyable { void fly(); }
class Bird implements Flyable {
    private final String species;
    Bird(String s) { species = s; }
    public void fly() { System.out.println(species + " flaps its wings and flies"); }
}
class Aeroplane implements Flyable {
    private final String model;
    Aeroplane(String m) { model = m; }
    public void fly() { System.out.println(model + " accelerates down the runway and takes off"); }
}
```

Output

```
"C:\Program Files\Java\jdk-26.0.2.1\bin\java.exe" "-jav
Eagle flaps its wings and flies
Boeing 737 accelerates down the runway and takes off
```

This is the case for an interface: a capability shared by unrelated classes.

Intermediate 1: Abstract Account with SavingsAccount and CurrentAccount

```
public class L6_Accounts {
   public static void main(String[] args) {
      Account[] accounts = { new SavingsAccount("S-1", 1000), new CurrentAccount("C-1", 1000)
};
      for (Account a : accounts) {
         a.deposit(500);
         a.applyMonthlyFee();
         System.out.println(a);
      }
   }
}
abstract class Account {
   private final String id; protected double balance;
   Account(String id, double opening) { this.id = id; this.balance = opening; }
   void deposit(double amt) {                    // concrete, shared by every account
      if (amt <= 0) throw new IllegalArgumentException("Deposit must be positive");
      balance += amt;
   }
   abstract double monthlyFee();                 // each type decides its own fee
   void applyMonthlyFee() { balance -= monthlyFee(); }
   @Override public String toString() { return String.format("%s [%s] fee %.2f balance %.2f",
getClass().getSimpleName(), id, monthlyFee(), balance); }
}
class SavingsAccount extends Account {
   SavingsAccount(String id, double o) { super(id, o); }
   double monthlyFee() { return 2.00; }
}
class CurrentAccount extends Account {
   CurrentAccount(String id, double o) { super(id, o); }
   double monthlyFee() { return 10.00; }
}
```

Output

```
"C:\Program Files\Java\jdk-26.0.2.1\bin\java.exe"
SavingsAccount [S-1] fee 2.00 balance 1498.00
CurrentAccount [C-1] fee 10.00 balance 1490.00
```

Intermediate 2: Student `Comparable` (by name) plus a `Comparator` (by marks)

```
import java.util.*;
public class L6_StudentSort {
   public static void main(String[] args) {
```

```
        List<Student> list = new ArrayList<>(List.of(new Student("Kuda", 61), new Student("Ada",
88), new Student("Lynn", 74)));
        Collections.sort(list);                                  // natural order: name (Comparable)
        System.out.println("By name:  " + list);
        list.sort(Comparator.comparingInt(Student::getMarks).reversed());    // alternative order:
marks (Comparator)
        System.out.println("By marks: " + list);
    }
}
class Student implements Comparable<Student> {
    private final String name; private final int marks;
    Student(String name, int marks) { this.name = name; this.marks = marks; }
    int getMarks() { return marks; }
    @Override public int compareTo(Student o) { return name.compareTo(o.name); }
    @Override public String toString() { return name + "(" + marks + ")"; }
}
```

Output

```
"C:\Program Files\Java\jdk-26.0.2.1\bin\java.exe"
By name:  [Ada(88), Kuda(61), Lynn(74)]
By marks: [Ada(88), Lynn(74), Kuda(61)]
```

A class has one natural ordering (`compareTo`) but can be sorted any number of other ways with external `Comparator` objects.

Conclusion. Abstract classes share code and state among closely related classes. Interfaces describe capabilities that unrelated classes can share. Both let calling code depend on the abstraction instead of the concrete class.

LAB7: Exception Handling

Objectives: `try`/`catch`/`finally`; custom checked exceptions; deciding between returning a value and throwing.

Basic 1: Read two integers and divide, handling bad input and division by zero

```
import java.util.*;
public class L7_Divide {
    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        try {
            System.out.print("Enter two integers: ");
            int a = Integer.parseInt(in.next());
            int b = Integer.parseInt(in.next());
            System.out.println(a + " / " + b + " = " + (a / b));
```

```
        } catch (NumberFormatException e) {        // non-numeric input
            System.out.println("Invalid input: please type whole numbers only");
        } catch (ArithmeticException e) {          // division by zero
            System.out.println("Cannot divide by zero");
        } catch (NoSuchElementException e) {       // fewer than two tokens typed
            System.out.println("Two numbers are required");
        } finally {
            in.close();
        }
    }
}
```

Output (three separate runs, typing `10 2`, then `10 0`, then `ten 5`)

```
"C:\Program Files\Java\jdk-26.0.2.1\bin\java.exe"
Enter two integers: 10 2
10 / 2 = 5
```

```
"C:\Program Files\Java\jdk-26.0.2.1\bin\java.exe"
Enter two integers: 10 0
Cannot divide by zero
```

```
"C:\Program Files\Java\jdk-26.0.2.1\bin\java.exe"
Enter two integers: 10 5
10 / 5 = 2
```

Basic 2: Array access with `ArrayIndexOutOfBoundsException` caught

```
public class L7_ArrayAccess {
    static int safeGet(int[] arr, int index) {
        try {
            return arr[index];
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Index " + index + " is invalid (valid: 0-" + (arr.length - 1) + ")");
            return -1;                          // sentinel value for "not found"
        }
    }
    public static void main(String[] args) {
        int[] data = {10, 20, 30};
        System.out.println("data[1] = " + safeGet(data, 1));
        System.out.println("data[7] = " + safeGet(data, 7));
        System.out.println("data[-1] = " + safeGet(data, -1));
```

```
    }
}
```

Output

```
"C:\Program Files\Java\jdk-26.0.2.1\bin\java.exe"
data[1] = 20
Index 7 is invalid (valid: 0-2)
data[7] = -1
Index -1 is invalid (valid: 0-2)
data[-1] = -1
```

Intermediate 1: Checked `InvalidAgeException` and `validateAge(int)`

```
public class L7_Age {
    static void validateAge(int age) throws InvalidAgeException {
        if (age < 0 || age > 120) throw new InvalidAgeException(age);
        System.out.println("Age " + age + " is valid");
    }
    public static void main(String[] args) {
        int[] tests = {25, 0, 120, -5, 150};
        for (int t : tests) {
            try { validateAge(t); }
            catch (InvalidAgeException e) { System.out.println("Rejected: " + e.getMessage()); }
        }
    }
}
class InvalidAgeException extends Exception {          // checked: callers MUST handle it
    private final int age;
    InvalidAgeException(int age) { super("Age must be between 0 and 120 but was " + age);
this.age = age; }
    int getAge() { return age; }
}
```

Output

```
"C:\Program Files\Java\jdk-26.0.2.1\bin\java.exe" "-ja
Age 25 is valid
Age 0 is valid
Age 120 is valid
Rejected: Age must be between 0 and 120 but was -5
Rejected: Age must be between 0 and 120 but was 150
```

The boundary values 0 and 120 are accepted, and -5 and 150 are rejected.

Intermediate 2: `withdraw` throws `InsufficientFundsException` instead of returning `false`

```
public class L7_BankThrows {
   public static void main(String[] args) {
      BankAccount acc = new BankAccount(500);
      try {
         acc.withdraw(100);
         System.out.println("Balance: " + acc.getBalance());
         acc.withdraw(5000);                    // caller cannot ignore the failure now
         System.out.println("never reached");
      } catch (InsufficientFundsException e) {
         System.out.println(e.getMessage() + " (short by " + e.getShortfall() + ")");
      } finally {
         System.out.println("Final balance: " + acc.getBalance());
      }
   }
}
class InsufficientFundsException extends Exception {
   private final double shortfall;
   InsufficientFundsException(String msg, double shortfall) { super(msg); this.shortfall = shortfall;
}
   double getShortfall() { return shortfall; }
}
class BankAccount {
   private double balance;
   BankAccount(double b) { balance = b; }
   double getBalance() { return balance; }
   void withdraw(double amt) throws InsufficientFundsException {
      if (amt <= 0) throw new IllegalArgumentException("Withdrawal must be positive");
      if (amt > balance) throw new InsufficientFundsException("Cannot withdraw " + amt + ";
balance is " + balance, amt - balance);
      balance -= amt;                          // state changes only after every check has passed
   }
}
```

Output

```
"C:\Program Files\Java\jdk-26.0.2.1\bin\java.exe" "-javaagent:C:\
Balance: 400.0
Cannot withdraw 5000.0; balance is 400.0 (short by 4600.0)
Final balance: 400.0
```

Trade-offs.

	Returning `boolean` (Lab 3)	Throwing a checked exception (this version)
Can the caller ignore failure?	Yes, silently, and the program carries on as if it worked	No, the compiler forces a `try`/`catch` or `throws`
Information about the failure	Only true/false	Message and extra data such as the shortfall
Code at the call site	Short	More verbose
Cost	Cheap	Creating an exception is slower
Best when	Failure is expected and routine	Failure is exceptional and the caller needs to react

The account state is unchanged when the exception is thrown, because every check happens before `balance` is modified.

Conclusion. Exceptions separate normal logic from error handling and force callers to deal with recoverable failures. Custom exceptions can carry useful data.

LAB8: Generics and Collections

Objectives: use `List`, `Set` and `Map`; write a generic class; sort with a custom comparator.

Basic 1: Ten integers in an `ArrayList`: sum, maximum, reversed

```
import java.util.*;
public class L8_IntList {
   public static void main(String[] args) {
      List<Integer> nums = new ArrayList<>(List.of(4, 18, 7, 25, 9, 12, 3, 30, 16, 1));
      int sum = 0, max = nums.get(0);
      for (int n : nums) { sum += n; if (n > max) max = n; }
      System.out.println("List:    " + nums);
      System.out.println("Sum:     " + sum);
      System.out.println("Maximum: " + max);
      List<Integer> reversed = new ArrayList<>(nums);   // copy so the original order is kept
      Collections.reverse(reversed);
      System.out.println("Reverse: " + reversed);
   }
}
```

Output

```
"C:\Program Files\Java\jdk-26.0.2.1\bin\java.exe"
List:    [4, 18, 7, 25, 9, 12, 3, 30, 16, 1]
Sum:     125
Maximum: 30
Reverse: [1, 16, 30, 3, 12, 9, 25, 7, 18, 4]
```

Basic 2: Remove duplicates from a list of names using a `Set`

```
import java.util.*;
public class L8_Dedupe {
   public static void main(String[] args) {
      List<String> names = List.of("Zodwa", "Alice", "Moyo", "Alice", "Zodwa", "Brian");
      Set<String> unique = new LinkedHashSet<>(names);   // no duplicates, keeps first-seen
order
      System.out.println("Original: " + names);
      System.out.println("Unique:   " + unique);
      System.out.println("Sorted:   " + new TreeSet<>(names));
   }
}
```

Output

```
"C:\Program Files\Java\jdk-26.0.2.1\bin\java.exe" "-
Original: [Zodwa, Alice, Moyo, Alice, Zodwa, Brian]
Unique:   [Zodwa, Alice, Moyo, Brian]
Sorted:   [Alice, Brian, Moyo, Zodwa]
```

`LinkedHashSet` keeps the first-seen order, while `TreeSet` sorts the names.

Intermediate 1: Generic `Stack<T>` backed by an `ArrayList<T>`

```
import java.util.*;
public class L8_GenericStack {
   public static void main(String[] args) {
      Stack<String> s = new Stack<>();
      s.push("a"); s.push("b"); s.push("c");
      System.out.println("peek = " + s.peek() + ", size = " + s.size());
      System.out.println("pop  = " + s.pop());
      System.out.println("pop  = " + s.pop());
      System.out.println("empty? " + s.isEmpty());
      Stack<Integer> nums = new Stack<>();          // same class, different type
      nums.push(10); nums.push(20);
      int top = nums.pop();                         // no cast needed
      System.out.println("Integer stack popped " + top);
```

```
        s.pop();
        try { s.pop(); }
        catch (IllegalStateException e) { System.out.println("Error: " + e.getMessage()); }
    }
}
class Stack<T> {
    private final List<T> items = new ArrayList<>();     // composition: stack HAS-A list
    void push(T item) { items.add(item); }
    T pop() {
        if (isEmpty()) throw new IllegalStateException("Stack is empty");
        return items.remove(items.size() - 1);
    }
    T peek() {
        if (isEmpty()) throw new IllegalStateException("Stack is empty");
        return items.get(items.size() - 1);
    }
    boolean isEmpty() { return items.isEmpty(); }
    int size() { return items.size(); }
}
```

Output

```
"C:\Program Files\Java\jdk-26.0.2.1\bin\java.exe"
peek = c, size = 3
pop  = c
pop  = b
empty? false
Integer stack popped 20
Error: Stack is empty
```

Intermediate 2: Word frequency, sorted by frequency descending

```
import java.util.*;
public class L8_WordFreq {
    public static void main(String[] args) {
        String text = "the cat and the dog and the bird saw the cat";
        Map<String, Integer> freq = new HashMap<>();
        for (String w : text.toLowerCase().split("\\s+")) freq.merge(w, 1, Integer::sum);

        List<Map.Entry<String, Integer>> entries = new ArrayList<>(freq.entrySet());
        // frequency descending, ties broken alphabetically
        entries.sort(Map.Entry.<String,
Integer>comparingByValue().reversed().thenComparing(Map.Entry.comparingByKey()));
        for (Map.Entry<String, Integer> e : entries) System.out.println(e.getKey() + " = " +
e.getValue());
```

```
        }
    }
```

Output

```
"C:\Program Files\Java\jdk-26.0.2.1\bin\java.exe"
the = 4
and = 2
cat = 2
bird = 1
dog = 1
saw = 1
```

A `Map` counts the words. A `HashMap` has no useful order, so the entries are copied into a list and sorted by value (descending) and then by key.

Conclusion. Choose the collection for the job (`List` for sequences, `Set` for uniqueness, `Map` for lookup), program to the interface, and use generics so type errors appear at compile time.

LAB9: Nested Types, Enums, Records and the Object Contract

Objectives: `enum` with `switch`; `record`; `equals`/`hashCode`; static nested and inner classes.

Basic 1: `enum Level` with a `switch`

```
public class L9_Level {
    enum Level { LOW, MEDIUM, HIGH }
    static void advice(Level l) {
        switch (l) {
            case LOW -> System.out.println("LOW: relax, you are on track");
            case MEDIUM -> System.out.println("MEDIUM: keep revising each day");
            case HIGH -> System.out.println("HIGH: start now and ask for help");
        }
    }
    public static void main(String[] args) {
        for (Level l : Level.values()) advice(l);
        System.out.println(Level.valueOf("HIGH") == Level.HIGH);   // == is correct for enums
    }
}
```

Output

```
"C:\Program Files\Java\jdk-26.0.2.1\bin\java.exe" "-j
LOW: relax, you are on track
MEDIUM: keep revising each day
HIGH: start now and ask for help
true
```

Basic 2: `record Student(String name, int age)`

```
public class L9_Record {
   record Student(String name, int age) { }
   public static void main(String[] args) {
      Student a = new Student("Tariro", 20);
      Student b = new Student("Tariro", 20);
      Student c = new Student("Farai", 21);
      System.out.println(a);
      System.out.println("a.equals(b): " + a.equals(b) + ", a == b: " + (a == b));
      System.out.println("a.equals(c): " + a.equals(c));
      System.out.println("Component name() = " + a.name() + ", age() = " + a.age());
   }
}
```

Output

```
"C:\Program Files\Java\jdk-26.0.2.1\bin\java.exe"
Student[name=Tariro, age=20]
a.equals(b): true, a == b: false
a.equals(c): false
Component name() = Tariro, age() = 20
```

The record gives the constructor, accessors, `equals`, `hashCode` and `toString` automatically, and equality is by value.

Intermediate 1: `Book` with `equals`/`hashCode` by ISBN, proved with a `HashSet`

```
import java.util.*;
public class L9_BookEquals {
   public static void main(String[] args) {
      Book b1 = new Book("978-0134685991", "Effective Java");
      Book b2 = new Book("978-0134685991", "Effective Java (2nd copy)");  // same ISBN
      Book b3 = new Book("978-0132350884", "Clean Code");
      Set<Book> set = new HashSet<>();
      System.out.println("add b1: " + set.add(b1));
      System.out.println("add b2: " + set.add(b2) + "  (rejected: same ISBN)");
      System.out.println("add b3: " + set.add(b3));
      System.out.println("Set size: " + set.size());
      System.out.println("hash codes equal? " + (b1.hashCode() == b2.hashCode()));
   }
}
class Book {
   private final String isbn, title;
```

```
    Book(String isbn, String title) { this.isbn = isbn; this.title = title; }
    @Override public boolean equals(Object o) {         // Object parameter, so it really overrides
      if (this == o) return true;
      if (!(o instanceof Book other)) return false;
      return isbn.equals(other.isbn);                   // identity = ISBN only
    }
    @Override public int hashCode() { return Objects.hash(isbn); }   // uses the same field as
equals
    @Override public String toString() { return title + " [" + isbn + "]"; }
}
```

Output

```
"C:\Program Files\Java\jdk-26.0.2.1\bin\java.exe"
add b1: true
add b2: false  (rejected: same ISBN)
add b3: true
Set size: 2
hash codes equal? true
```

`add` returns `false` for the second copy, so the set holds only 2 books. This works because `equals` and `hashCode` are based on the same field (the ISBN).

Intermediate 2: Linked list with a static nested `Node` and an inner `Iterator`

```
import java.util.Iterator;
import java.util.NoSuchElementException;
public class L9_LinkedList {
    public static void main(String[] args) {
        MyList<String> list = new MyList<>();
        list.add("one"); list.add("two"); list.add("three");
        for (String s : list) System.out.println(s);      // works because MyList is Iterable
        System.out.println("size = " + list.size());
    }
}
class MyList<T> implements Iterable<T> {
    private static class Node<T> {                    // static nested: needs no MyList instance
        final T value; Node<T> next;
        Node(T value) { this.value = value; }
    }
    private Node<T> head, tail;
    private int size;
    void add(T value) {
        Node<T> n = new Node<>(value);
```

```
        if (head == null) head = tail = n; else { tail.next = n; tail = n; }
        size++;
    }
    int size() { return size; }
    @Override public Iterator<T> iterator() { return new ListIterator(); }
    private class ListIterator implements Iterator<T> {    // inner class: reads the outer 'head'
        private Node<T> current = head;
        public boolean hasNext() { return current != null; }
        public T next() {
            if (current == null) throw new NoSuchElementException();
            T v = current.value; current = current.next; return v;
        }
    }
}
```

Output

```
"C:\Program Files\Java\jdk-26.0.2.1\bin\java.exe"
one
two
three
size = 3
```

`Node` is a static nested class because it needs no reference to a particular list. `ListIterator` is an inner class because it must read the outer list's `head`.

Conclusion. Enums model fixed sets of constants and records model immutable data. `equals` and `hashCode` must be overridden together, and nested classes keep helper types next to the class that uses them.

LAB10: File I/O, Lambdas and Streams

Objectives: read and write files with `java.nio.file`; use lambdas and `Predicate`; process data with streams.

Basic 1: Save five typed lines to a file and read them back

```
import java.io.IOException;
import java.nio.file.*;
import java.util.*;
public class L10_SaveLines {
    public static void main(String[] args) {
        Path file = Path.of("typed-lines.txt");
        Scanner in = new Scanner(System.in);
        List<String> lines = new ArrayList<>();
        for (int i = 1; i <= 5; i++) {
```

```
            System.out.print("Line " + i + ": ");
            lines.add(in.nextLine());
        }
        try {
            Files.write(file, lines);
            System.out.println("--- saved, now reading back ---");
            for (String l : Files.readAllLines(file)) System.out.println(l);
        } catch (IOException e) {
            System.out.println("File error: " + e.getMessage());
        }
    }
}
```

Output

```
"C:\Program Files\Java\jdk-26.0.2.1\bin\java.exe"
Line 1: YELLOW GOOSE
Line 2: The world is my oyster
Line 3: Life is a wheel
Line 4: One step at a time
Line 5: One time one  time
--- saved, now reading back ---
YELLOW GOOSE
The world is my oyster
Life is a wheel
One step at a time
One time one  time
```

Basic 2: Delete all even numbers with a `Predicate` and `removeIf`

```
import java.util.*;
import java.util.function.Predicate;
public class L10_RemoveIf {
    public static void main(String[] args) {
        List<Integer> nums = new ArrayList<>(List.of(1, 2, 3, 4, 5, 6, 7, 8, 9, 10));
        Predicate<Integer> isEven = n -> n % 2 == 0;
        nums.removeIf(isEven);
        System.out.println(nums);
    }
}
```

Output

```
"C:\Program Files\Java\jdk-26.0.2.1\bin\java.e
[1, 3, 5, 7, 9]
```

`removeIf` is safe, whereas removing inside a for-each loop would throw `ConcurrentModificationException`.

Intermediate 1: Count lines, words and characters in a text file

```
import java.io.IOException;
import java.nio.file.*;
import java.util.List;
public class L10_WordCount {
   public static void main(String[] args) {
      Path file = Path.of("sample.txt");
      try {
         Files.writeString(file, "Object oriented programming\nis built on classes\nand
objects.\n");   // create sample input
         List<String> lines = Files.readAllLines(file);
         int words = 0, chars = 0;
         for (String line : lines) {
            if (!line.isBlank()) words += line.trim().split("\\s+").length;
            chars += line.length();                    // characters excluding line breaks
         }
         System.out.println("Lines:      " + lines.size());
         System.out.println("Words:      " + words);
         System.out.println("Characters: " + chars);
      } catch (IOException e) {
         System.out.println("Could not read file: " + e.getMessage());
      }
   }
}
```

Output

```
"C:\Program Files\Java\jdk-26.0.2.1\bin\java.exe"
Lines:      3
Words:      9
Characters: 58
```

The program writes its own sample file first so it runs anywhere. To count another file, change the `Path`. Characters exclude the line breaks.

Intermediate 2: Students above the class average, sorted alphabetically (streams)

```
import java.util.*;
public class L10_AboveAverage {
   record Student(String name, double marks) { }
   public static void main(String[] args) {
      List<Student> class1 = List.of(new Student("Kuda", 61), new Student("Ada", 88), new
```

```
        Student("Lynn", 74),
                                new Student("Ben", 52), new Student("Zodwa", 79));
        double avg = class1.stream().mapToDouble(Student::marks).average().orElse(0);
        System.out.println("Class average: " + avg);
        class1.stream()
            .filter(s -> s.marks() > avg)            // above average only
            .map(Student::name)
            .sorted()                          // alphabetical
            .forEach(System.out::println);
      }
    }
```

Output

```
"C:\Program Files\Java\jdk-26.0.2.1\bin\java.exe"
Class average: 70.8
Ada
Lynn
Zodwa
```

The pipeline is a source (the list), lazy intermediate steps (`filter`, `map`, `sorted`) and a terminal operation (`forEach`) that triggers the work.

Conclusion. `java.nio.file` with `IOException` handling gives simple file persistence. Lambdas and method references supply short implementations of functional interfaces, and streams express filter/map/sort pipelines declaratively without changing the source.

Part 11: Mini-Project – Library Management System

This is the Lab 11 mini-project (30% of the course). It compiles and runs as shown below, and the test program passes all 15 checks. The files go together in one folder and are compiled with `javac *.java` (run `java LibraryTest` for the tests and `java LibraryApp` for the menu).

Design

```
          <<interface>>
           Borrowable
       borrow(Member), giveBack()
             ^
             | implements
      <<abstract>> LibraryItem  ---- id, title, available
      loanPeriodDays(), toCsv()
        ^           ^
        |           |
       Book         Dvd
    (21-day loan)   (7-day loan)
```

Member ---- id, name, borrowedItemIds (max 3)
Loan ---- record: itemId, memberId, borrowedOn, dueOn

Library HAS Map<String,LibraryItem>, Map<String,Member>, List<Loan>
borrow / giveBack / overdue / searchByTitle / save / load

ItemNotAvailableException, MemberLimitExceededException (custom, checked)
LibraryApp = menu (console UI) LibraryTest = checks

How the OOP principles are used

- **Encapsulation:** all fields are private, constructors validate their input, and getters hand out unmodifiable views (for example `Member.getBorrowedItemIds()`).
- **Inheritance and abstraction:** the abstract `LibraryItem` holds the shared id, title and availability and declares `loanPeriodDays()` and `toCsv()`. `Book` and `Dvd` extend it.
- **Polymorphism:** `Library` works with `LibraryItem` references, so `item.loanPeriodDays()` gives 21 days for a book and 7 for a DVD with no `if` statements.
- **Interface:** `Borrowable` is the contract for anything that can be lent. `LibraryItem` implements it.
- **Exceptions:** two custom checked exceptions for business-rule failures, and `IllegalArgumentException` for bad data.
- **Collections and streams:** `Map<String, ...>` keyed by id, and a stream pipeline for the title search and the overdue list.
- **File persistence:** items, members and loans are saved to three CSV files on exit and reloaded at start.

Exceptions and interface

```
// ItemNotAvailableException.java
public class ItemNotAvailableException extends Exception {
   public ItemNotAvailableException(String itemId) { super("Item " + itemId + " is already on
loan"); }
}

// MemberLimitExceededException.java
public class MemberLimitExceededException extends Exception {
   public MemberLimitExceededException(String memberId, int max) {
      super("Member " + memberId + " already has the maximum of " + max + " loans");
   }
}

// Borrowable.java
public interface Borrowable {
   void borrow(Member member) throws ItemNotAvailableException;   // contract: anything
lendable
```

```
    void giveBack();
}
```

LibraryItem, Book and Dvd

```
// LibraryItem.java
public abstract class LibraryItem implements Borrowable {
    private final String id;
    private final String title;
    private boolean available = true;

    protected LibraryItem(String id, String title) {
        if (id == null || id.isBlank()) throw new IllegalArgumentException("Item id required");
        if (title == null || title.isBlank()) throw new IllegalArgumentException("Title required");
        if (id.contains(",") || title.contains(",")) throw new IllegalArgumentException("Commas are
not allowed (CSV storage)");
        this.id = id.trim();
        this.title = title.trim();
    }

    public abstract int loanPeriodDays();      // each item type sets its own loan period
(polymorphism)
    public abstract String toCsv();           // each item type knows how to save itself

    @Override
    public void borrow(Member member) throws ItemNotAvailableException {
        if (!available) throw new ItemNotAvailableException(id);
        available = false;
    }

    @Override
    public void giveBack() { available = true; }

    public String getId() { return id; }
    public String getTitle() { return title; }
    public boolean isAvailable() { return available; }
    void setAvailable(boolean available) { this.available = available; }   // package-private: used
when reloading

    @Override
    public String toString() {
        return String.format("%-4s %-22s %-5s %s (%d-day loan)", id, title,
getClass().getSimpleName(),
                available ? "available" : "ON LOAN", loanPeriodDays());
    }
```

```
}

// Book.java
public class Book extends LibraryItem {
   private final String author;
   public Book(String id, String title, String author) {
      super(id, title);
      if (author == null || author.isBlank() || author.contains(",")) throw new
IllegalArgumentException("Valid author required");
      this.author = author.trim();
   }
   @Override public int loanPeriodDays() { return 21; }
   public String getAuthor() { return author; }
   @Override public String toCsv() { return "BOOK," + getId() + "," + getTitle() + "," + author; }
}

// Dvd.java
public class Dvd extends LibraryItem {
   private final int minutes;
   public Dvd(String id, String title, int minutes) {
      super(id, title);
      if (minutes <= 0) throw new IllegalArgumentException("Running time must be positive");
      this.minutes = minutes;
   }
   @Override public int loanPeriodDays() { return 7; }
   public int getMinutes() { return minutes; }
   @Override public String toCsv() { return "DVD," + getId() + "," + getTitle() + "," + minutes; }
}
```

Member and Loan

```
// Member.java
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class Member {
   public static final int MAX_LOANS = 3;
   private final String id;
   private String name;
   private final List<String> borrowedItemIds = new ArrayList<>();

   public Member(String id, String name) {
      if (id == null || id.isBlank()) throw new IllegalArgumentException("Member id required");
      if (name == null || name.isBlank() || name.contains(",")) throw new
```

```
IllegalArgumentException("Valid name required");
        this.id = id.trim();
        this.name = name.trim();
    }

    void addLoan(String itemId) throws MemberLimitExceededException {
        if (borrowedItemIds.size() >= MAX_LOANS) throw new MemberLimitExceededException(id,
MAX_LOANS);
        borrowedItemIds.add(itemId);
    }
    void removeLoan(String itemId) { borrowedItemIds.remove(itemId); }

    public String getId() { return id; }
    public String getName() { return name; }
    public List<String> getBorrowedItemIds() { return
Collections.unmodifiableList(borrowedItemIds); }   // encapsulation
    @Override public String toString() { return id + " " + name + " (" + borrowedItemIds.size() + "/"
+ MAX_LOANS + " loans)"; }
}

// Loan.java
import java.time.LocalDate;

public record Loan(String itemId, String memberId, LocalDate borrowedOn, LocalDate dueOn) {
    public boolean isOverdue(LocalDate today) { return today.isAfter(dueOn); }
    public String toCsv() { return itemId + "," + memberId + "," + borrowedOn + "," + dueOn; }
    public static Loan fromCsv(String line) {
        String[] p = line.split(",");
        return new Loan(p[0], p[1], LocalDate.parse(p[2]), LocalDate.parse(p[3]));
    }
}
```

Library (core service and persistence)

```
// Library.java
import java.io.IOException;
import java.nio.file.*;
import java.time.LocalDate;
import java.util.*;

public class Library {
    private final Map<String, LibraryItem> items = new HashMap<>();
    private final Map<String, Member> members = new HashMap<>();
    private final List<Loan> loans = new ArrayList<>();
```

```
  public void addItem(LibraryItem item) {
    if (items.putIfAbsent(item.getId(), item) != null) throw new
IllegalArgumentException("Duplicate item id " + item.getId());
  }
  public void addMember(Member m) {
    if (members.putIfAbsent(m.getId(), m) != null) throw new
IllegalArgumentException("Duplicate member id " + m.getId());
  }

  public void borrow(String memberId, String itemId) throws ItemNotAvailableException,
MemberLimitExceededException {
    borrow(memberId, itemId, LocalDate.now());
  }

  public void borrow(String memberId, String itemId, LocalDate today)
      throws ItemNotAvailableException, MemberLimitExceededException {
    Member member = members.get(memberId);                    // 1. look up both
    LibraryItem item = items.get(itemId);
    if (member == null) throw new IllegalArgumentException("No such member: " + memberId);
    if (item == null) throw new IllegalArgumentException("No such item: " + itemId);
    if (!item.isAvailable()) throw new ItemNotAvailableException(itemId);    // 2. check
availability and limit
    member.addLoan(itemId);                                  //    (throws if member is full)
    try {
      item.borrow(member);                                  // 3. record everywhere
    } catch (ItemNotAvailableException e) {
      member.removeLoan(itemId);                            // roll back so state stays
consistent
      throw e;
    }
    loans.add(new Loan(itemId, memberId, today, today.plusDays(item.loanPeriodDays())));
  }

  public void giveBack(String itemId) {
    Loan loan = loans.stream().filter(l -> l.itemId().equals(itemId)).findFirst()
        .orElseThrow(() -> new IllegalArgumentException("Item " + itemId + " is not on loan"));
    loans.remove(loan);
    items.get(itemId).giveBack();
    members.get(loan.memberId()).removeLoan(itemId);
  }

  public List<Loan> overdue(LocalDate today) {
    return loans.stream().filter(l ->
l.isOverdue(today)).sorted(Comparator.comparing(Loan::dueOn)).toList();
```

```
    }

    public List<LibraryItem> searchByTitle(String text) {
        return items.values().stream()
                .filter(i -> i.getTitle().toLowerCase().contains(text.toLowerCase()))
                .sorted(Comparator.comparing(LibraryItem::getTitle))
                .toList();
    }

    public Collection<LibraryItem> getItems() { return
Collections.unmodifiableCollection(items.values()); }
    public Collection<Member> getMembers() { return
Collections.unmodifiableCollection(members.values()); }
    public List<Loan> getLoans() { return Collections.unmodifiableList(loans); }

    // ---------- persistence (Lab 10): three CSV files ----------
    public void save(Path dir) throws IOException {
        Files.createDirectories(dir);
        Files.write(dir.resolve("items.csv"),
items.values().stream().map(LibraryItem::toCsv).sorted().toList());
        Files.write(dir.resolve("members.csv"), members.values().stream().map(m -> m.getId() + ","
+ m.getName()).sorted().toList());
        Files.write(dir.resolve("loans.csv"), loans.stream().map(Loan::toCsv).toList());
    }

    public static Library load(Path dir) throws IOException {
        Library lib = new Library();
        if (!Files.exists(dir.resolve("items.csv"))) return lib;              // first run: nothing saved yet
        for (String line : Files.readAllLines(dir.resolve("items.csv"))) {
            if (line.isBlank()) continue;
            String[] p = line.split(",");
            lib.addItem(p[0].equals("BOOK") ? new Book(p[1], p[2], p[3]) : new Dvd(p[1], p[2],
Integer.parseInt(p[3])));
        }
        for (String line : Files.readAllLines(dir.resolve("members.csv"))) {
            if (line.isBlank()) continue;
            String[] p = line.split(",");
            lib.addMember(new Member(p[0], p[1]));
        }
        for (String line : Files.readAllLines(dir.resolve("loans.csv"))) {       // rebuild availability from
the loans
            if (line.isBlank()) continue;
            Loan loan = Loan.fromCsv(line);
            lib.loans.add(loan);
```

```
            lib.items.get(loan.itemId()).setAvailable(false);
            try { lib.members.get(loan.memberId()).addLoan(loan.itemId()); }
            catch (MemberLimitExceededException e) { throw new IOException("Corrupt data: " +
    e.getMessage(), e); }
        }
        return lib;
    }
}
```

If the member has no free slot, `borrow` fails before any state changes. If the item turns out to be unavailable after the member's slot was reserved, the slot is released again, so the data never ends up half-updated.

LibraryApp (menu)

```
// LibraryApp.java
import java.io.IOException;
import java.nio.file.Path;
import java.time.LocalDate;
import java.util.List;
import java.util.Scanner;

public class LibraryApp {
    private static final Scanner in = new Scanner(System.in);

    private static String ask(String prompt) { System.out.print(prompt); return in.nextLine().trim();
}

    public static void main(String[] args) throws IOException {
        Path dir = Path.of(args.length > 0 ? args[0] : "library-data");
        Library lib = Library.load(dir);
        System.out.println("Loaded " + lib.getItems().size() + " items, " + lib.getMembers().size()
                + " members, " + lib.getLoans().size() + " loans");

        boolean running = true;
        while (running) {
            System.out.println("\n1 Add item  2 Register member  3 Borrow  4 Return  5 Overdue  6
    Search  7 List all  0 Save & exit");
            String choice = ask("> ");
            try {
                switch (choice) {
                    case "1" -> {
                        String type = ask("Type (book/dvd): "), id = ask("Id: "), title = ask("Title: ");
                        if (type.equalsIgnoreCase("book")) lib.addItem(new Book(id, title, ask("Author: ")));
                        else lib.addItem(new Dvd(id, title, Integer.parseInt(ask("Minutes: "))));
                        System.out.println("Added " + id);
```

```
                }
                case "2" -> { lib.addMember(new Member(ask("Member id: "), ask("Name: "))); System.out.println("Registered"); }
                case "3" -> { lib.borrow(ask("Member id: "), ask("Item id: ")); System.out.println("Borrowed"); }
                case "4" -> { lib.giveBack(ask("Item id: ")); System.out.println("Returned"); }
                case "5" -> {
                    List<Loan> late = lib.overdue(LocalDate.now());
                    if (late.isEmpty()) System.out.println("No overdue items");
                    late.forEach(l -> System.out.println("OVERDUE: " + l.itemId() + " (member " + l.memberId() + ", due " + l.dueOn() + ")"));
                }
                case "6" -> {
                    List<LibraryItem> found = lib.searchByTitle(ask("Search text: "));
                    if (found.isEmpty()) System.out.println("No matches");
                    found.forEach(System.out::println);
                }
                case "7" -> {
lib.getItems().stream().sorted(java.util.Comparator.comparing(LibraryItem::getId)).forEach(System.out::println);
                        lib.getMembers().forEach(System.out::println); }
                case "0" -> running = false;
                default -> System.out.println("Unknown option");
            }
        } catch (ItemNotAvailableException | MemberLimitExceededException e) {
            System.out.println("Cannot borrow: " + e.getMessage());          // business rule failures
        } catch (IllegalArgumentException e) {
            System.out.println("Invalid input: " + e.getMessage());          // bad data
        }
    }
    lib.save(dir);
    System.out.println("Data saved to " + dir + ". Goodbye.");
  }
}
```

Testing (LibraryTest)

```
// LibraryTest.java
import java.nio.file.*;
import java.time.LocalDate;

// Tiny self-contained test runner (no JUnit needed)
public class LibraryTest {
  static int passed = 0, failed = 0;
  static void check(String name, boolean ok) {
```

```
        System.out.println((ok ? "PASS  " : "FAIL  ") + name);
        if (ok) passed++; else failed++;
    }
    static Library fresh() {
        Library lib = new Library();
        lib.addItem(new Book("B1", "Java Basics", "A. Author"));
        lib.addItem(new Book("B2", "Advanced Java", "B. Writer"));
        lib.addItem(new Book("B3", "Data Structures", "C. Coder"));
        lib.addItem(new Dvd("D1", "Java Lectures", 90));
        lib.addMember(new Member("M1", "Tariro"));
        lib.addMember(new Member("M2", "Farai"));
        return lib;
    }

    public static void main(String[] args) throws Exception {
        Library lib = fresh();

        lib.borrow("M1", "B1");
        check("borrow marks item unavailable", !lib.searchByTitle("Java
Basics").get(0).isAvailable());

        boolean blocked = false;
        try { lib.borrow("M2", "B1"); } catch (ItemNotAvailableException e) { blocked = true; }
        check("borrowing an item already on loan throws ItemNotAvailableException", blocked);

        lib.borrow("M1", "B2"); lib.borrow("M1", "B3");
        boolean limited = false;
        try { lib.borrow("M1", "D1"); } catch (MemberLimitExceededException e) { limited = true; }
        check("4th loan throws MemberLimitExceededException", limited);
        check("failed borrow leaves DVD available",
lib.searchByTitle("Lectures").get(0).isAvailable());

        lib.giveBack("B1");
        check("return makes item available again", lib.searchByTitle("Java
Basics").get(0).isAvailable());
        check("return frees a slot for the member", lib.getMembers().stream().filter(m ->
m.getId().equals("M1")).findFirst().get().getBorrowedItemIds().size() == 2);

        check("search is case-insensitive and sorted by title",

lib.searchByTitle("JAVA").stream().map(LibraryItem::getTitle).toList().equals(java.util.List.of("Ad
vanced Java", "Java Basics", "Java Lectures")));

        check("books due in 21 days, DVDs in 7 (polymorphism)", new Book("X", "t",
```

```
"a").loanPeriodDays() == 21 && new Dvd("Y", "t", 10).loanPeriodDays() == 7);

        check("no loans overdue today", lib.overdue(LocalDate.now()).isEmpty());
        check("both open loans overdue 30 days from now",
    lib.overdue(LocalDate.now().plusDays(30)).size() == 2);

        boolean bad = false;
        try { new Dvd("D9", "Bad", -5); } catch (IllegalArgumentException e) { bad = true; }
        check("invalid data rejected by constructor", bad);

        Path dir = Files.createTempDirectory("lib-test");
        lib.save(dir);
        Library copy = Library.load(dir);
        check("persistence: items survive save/load", copy.getItems().size() == 4);
        check("persistence: loans survive save/load", copy.getLoans().size() == 2);
        check("persistence: availability rebuilt from loans",
    !copy.searchByTitle("Advanced").get(0).isAvailable());
        boolean stillBlocked = false;
        try { copy.borrow("M2", "B2"); } catch (ItemNotAvailableException e) { stillBlocked = true; }
        check("rules still enforced after reload", stillBlocked);

        System.out.println("\n" + passed + " passed, " + failed + " failed");
      }
    }
```

Test output

```
"C:\Program Files\Java\jdk-26.0.2.1\bin\java.exe" "-javaagent:C:\Program Files\JetBra
PASS  borrow marks item unavailable
PASS  borrowing an item already on loan throws ItemNotAvailableException
PASS  4th loan throws MemberLimitExceededException
PASS  failed borrow leaves DVD available
PASS  return makes item available again
PASS  return frees a slot for the member
PASS  search is case-insensitive and sorted by title
PASS  books due in 21 days, DVDs in 7 (polymorphism)
PASS  no loans overdue today
PASS  both open loans overdue 30 days from now
PASS  invalid data rejected by constructor
PASS  persistence: items survive save/load
PASS  persistence: loans survive save/load
PASS  persistence: availability rebuilt from loans
PASS  rules still enforced after reload

15 passed, 0 failed
```

program restarted, data reloaded from the CSV files

```
"C:\Program Files\Java\jdk-26.0.2.1\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Communi
Loaded 4 items, 2 members, 1 loans

1 Add item  2 Register member  3 Borrow  4 Return  5 Overdue  6 Search  7 List all  0 Save & exit
> 7
B1   Java Basics            Book  available (21-day loan)
B2   Advanced Java          Book  available (21-day loan)
B3   Aqua                   Book  ON LOAN (21-day loan)
D1   Java for Biginners     Book  available (21-day loan)
Innocent Muchada M1 (0/3 loans)
M2 Tavonga (1/3 loans)

1 Add item  2 Register member  3 Borrow  4 Return  5 Overdue  6 Search  7 List all  0 Save & exit
```

The due dates are 21 days after the loan date, as Books specify.